

Lecture 4

p1 due tonight
p2 monday

Advanced Scheduling

CS 537, Spring 2026

Mike Swift & Tej Chajed

Life is not fair. Only the Completely Fair Scheduler is fair.



Would you rather:

- play the world's instruments expertly
- speak the world's languages fluently

Writing assignment

Read *The Machine Stops* by E.M. Forster (1909)

Write a reflection

See Canvas for details

Learning outcomes

Compare and contrast basic scheduling (e.g., MLFQ) with fair scheduling (e.g., CFS)

Analyze limitations of fair scheduling

Articulate the challenges of multiprocessor scheduling

Goal: proportional share scheduling

Goal: give each of N processes $1/N^{\text{th}}$ of the time on a CPU

- 1 process: 100% of CPU
- 3 processes: each get 1/3rd of CPU

Answers *how much* time a process gets but not *when* a process runs

Fairness: each process gets the same amount

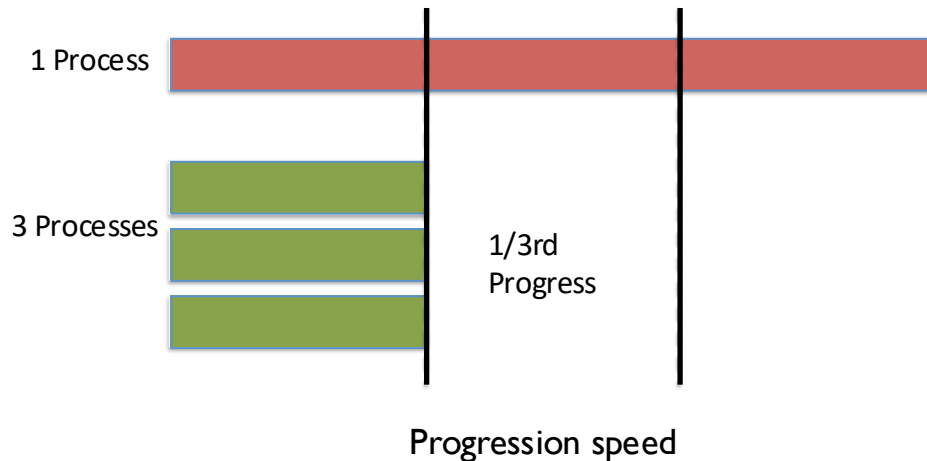
Contrast with MLFQ's goals

Completely Fair Scheduler - Concept

CFS

Idea: give each of N processes $1/N^{\text{th}}$ of the time on a CPU

- Consider ideal multitasking CPU with Infinitesimally small timeslice
 - Context switch instantly after each instruction
- N processes: each progresses uniformly at $1/N^{\text{th}}$ rate



Completely Fair Scheduler Design

Goal: give each of N processes $1/N^{\text{th}}$ of the time on a CPU

How (generally):

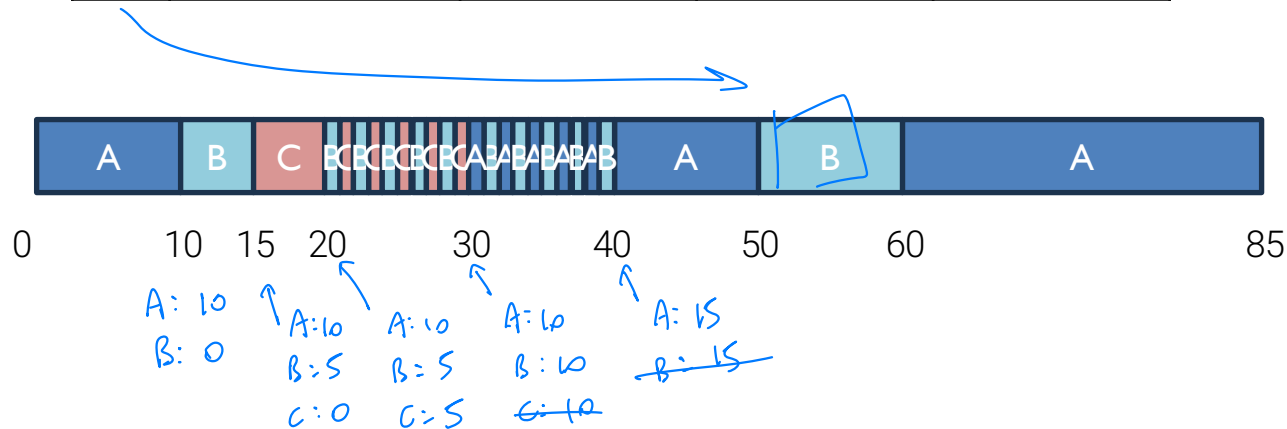
- Keep track of CPU time usage of a process (*runtime*)
- Always run the process with the lowest runtime

How it works:

- I/O bound processes haven't run in a while, so they are behind and get to catch up quickly
 - Low response time
- CPU bound processes have run recently, so they get preempted

Fair Scheduling Example

Job	Arrival(s)	run time (s)	Runtime value	Completion time
A	0	50		
B	10	15		
C	15	10		
B	50	10		



Fair Scheduling Example

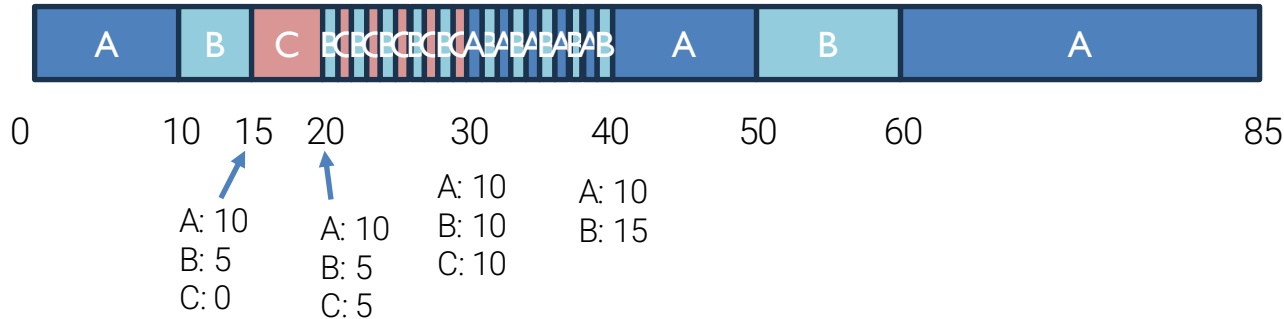
Job	Arrival(s)	run time (s)	Runtime value	Completion time
A	0	50		85
B	10	15		40
C	15	10		30
B	50	10		60

Average
Turnaround
Time

$$\frac{(85-0)+(40-10)+(30-15)+(60-50)}{4} = 35$$

Average
Response
Time

0



Problems

Frequent context switches adds overhead $\sim 1\mu s$ each

Equal isn't always best: **weight** processes differently

Long sleeping tasks: what runtime value when waking up?

Reducing context switches

Context switching frequently is inefficient

Solution: *Maximum overrun, C*

- How much ahead of lowest-time thread can you get, C ?
- Run process P_a if $P_a.\text{vruntime} < P_b.\text{vruntime} - C$

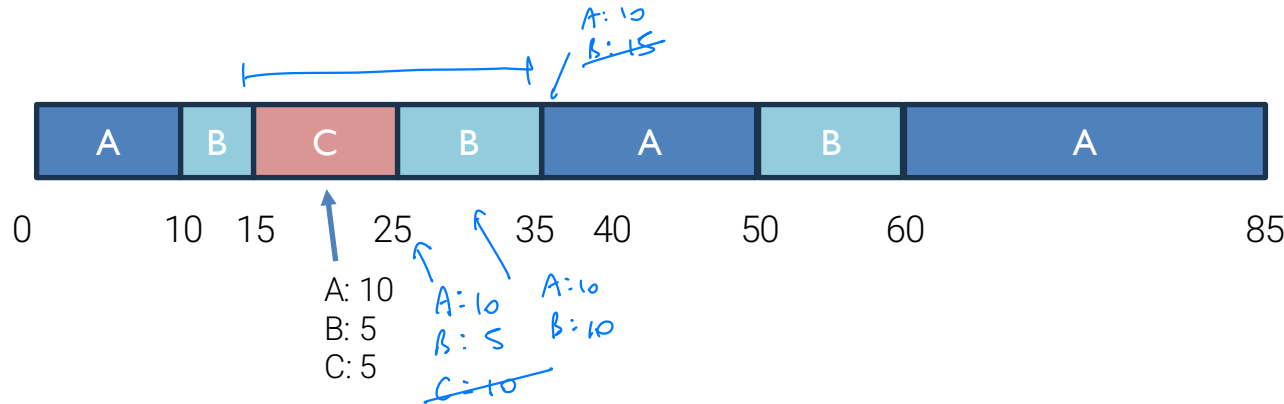
Equivalent to a minimum *timeslice*

Fair Scheduling Example with C=5

Job	Arrival(s)	run time (s)	Runtime value	Completion time
A	0	50		85
B	10	15		35
C	15	10		25 25
B	50	10		60

→

25 - 15



Average
Turnaround
Time

$$\frac{(85-0) + (35-10) + (25-15) + (60-50)}{4} = 32.5$$

Average
Response
Time

0

Weighted Fair Scheduling

What if some processes are more important than others?

- Add *weight* to each process. More weight → more runtime
- Processes run in proportion to weights
 - A: 4; B: 2, C:1 → runtime proportions 4:2:1

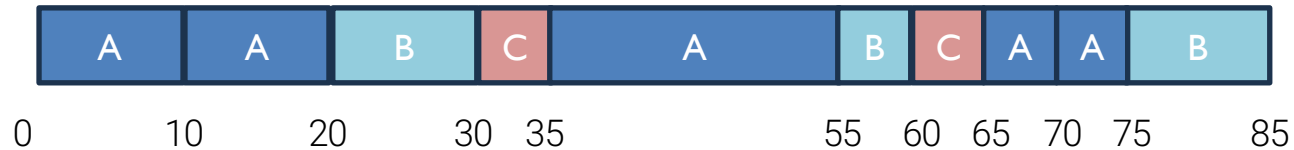
Calculate *virtual runtime* = real runtime / weight

- Run the process with the lowest virtual runtime

Overrun uses virtual runtime

Weighted Fair Scheduling Example, C=5

Job	Arrival(s)	run time (s)	Weight	Virtual Runtime
A	0	50	4	
B	10	15	2	
C	15	10	1	
B	70	10	2	



Graphical view

Virtual runtime for a process advances when runs

- Shallower for processes with larger weight
- Steeper for lower weight

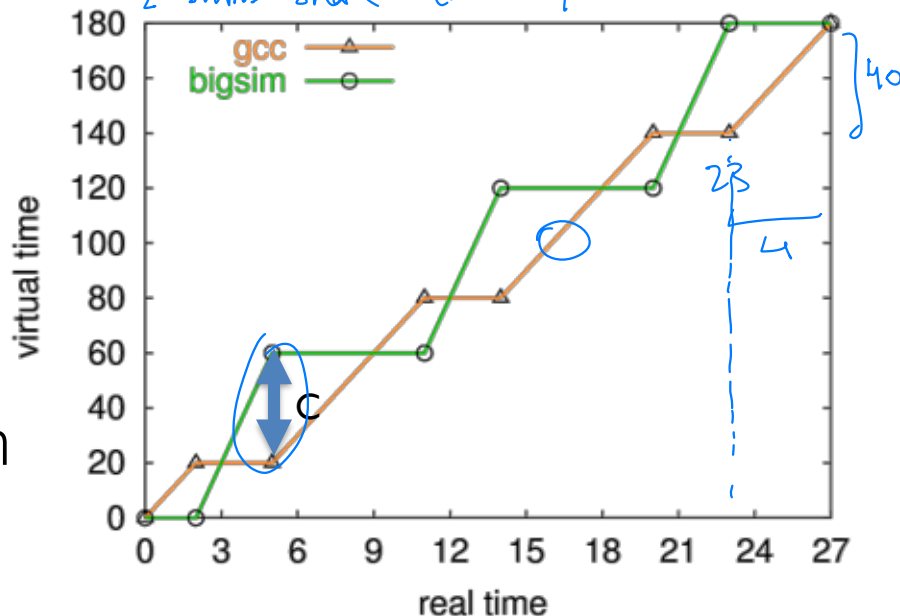
Step shows maximum overrun

$$vruntime = runtime / weight$$

What are the weights of those two processes (gcc, bigsim)?

1mm: think

2-3mm: share (with a partner)

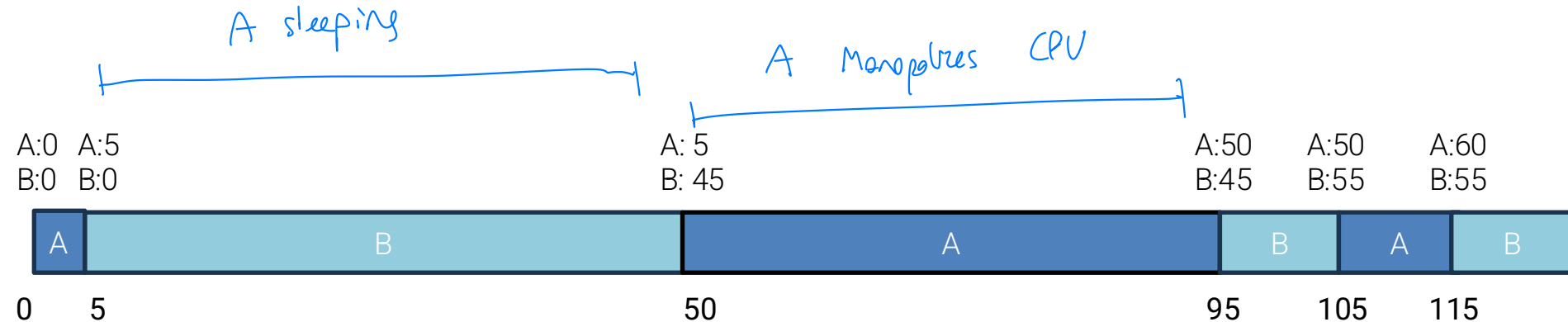


Problem: long-sleeping tasks

What happens when a sleeping task wakes?

A sleeps 45s →

Job	Arrival(s)	run time (s)	Weight
A	0	5	1
B	5	100	1
A	50	50	1



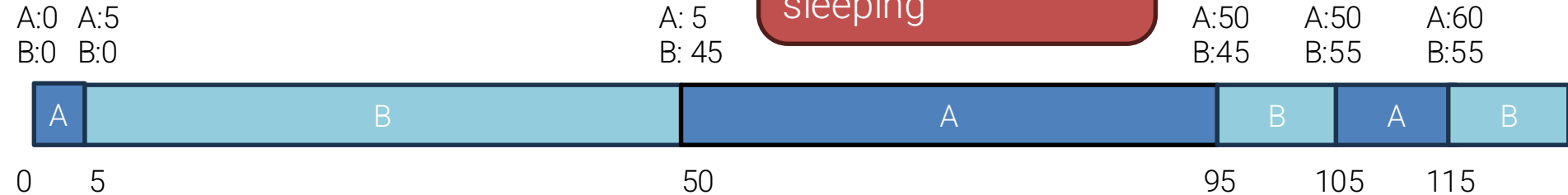
Problem: long-sleeping tasks

What happens when a sleeping task wakes?

A sleeps 45s →

Job	Arrival(s)	run time (s)	Weight
A	0	5	1
B	5	100	1
A	50	50	1

A monopolizes CPU because it was sleeping



Solution: reset vruntime

Calculate minimum vruntime for all processes

running ✓

If $P.vruntime > \text{minimum} \rightarrow$ do nothing

- Short sleep

If $P.vruntime < \text{minimum} \rightarrow$ set
 $P.vruntime = \text{minimum} - \text{threshold } D$

- D : preemption threshold ($\geq C$)

Job	Arrival(s)	run time (s)	Weight
A	0	5	1
B	5	100	1
A	50	50	1

A:0 A:5
B:0 B:0

A: 40 A:50 A:50 A:60
B: 45 B:45 B:55 B:55



$D = C$ shown

0 5 50 60 70 80

Quiz 4

<https://go.wisc.edu/c70w4c>



A better approach?

scx_horoscope - Astrological CPU Scheduler

"Why let mere mortals decide CPU priorities when the cosmos can guide us?"

A fully functional sched_ext scheduler that makes real CPU scheduling decisions based on real-time planetary positions, zodiac signs, and astrological principles. This actually loads into the Linux kernel and schedules your system tasks. Because if the universe can influence our lives, why not our CPU scheduling too?

Features

- **Real Planetary Calculations:** Uses the `astro` crate for accurate geocentric planetary positions
- **Zodiac-Based Task Classification:** Tasks are classified by their astrological affinities
- **Retrograde Detection:** Real retrograde motion detection by comparing day-to-day positions - negative influences trigger 50% time slice penalties
- **Lunar Phase Scheduling:** Moon phases affect Interactive tasks (shells, editors) with Full Moon giving 1.4x boost
- **Element Boosts & Debuffs:** Fire signs boost CPU tasks (1.5x), Water signs debuff them (0.6x) - elemental oppositions create cosmic chaos
- **Cosmic Weather Reports:** Get real-time astrological guidance for your system with moon phase tracking
- **Actually Works:** Loads into the Linux kernel via sched_ext and schedules real system processes
- **Real BPF Integration:** Uses `scx_rustland_core` framework for kernel-userspace communication
- **Dynamic Time Slicing:** Adjusts CPU time based on astrological priority (100-1000)

Status Check

Thus far, assumed a single CPU core

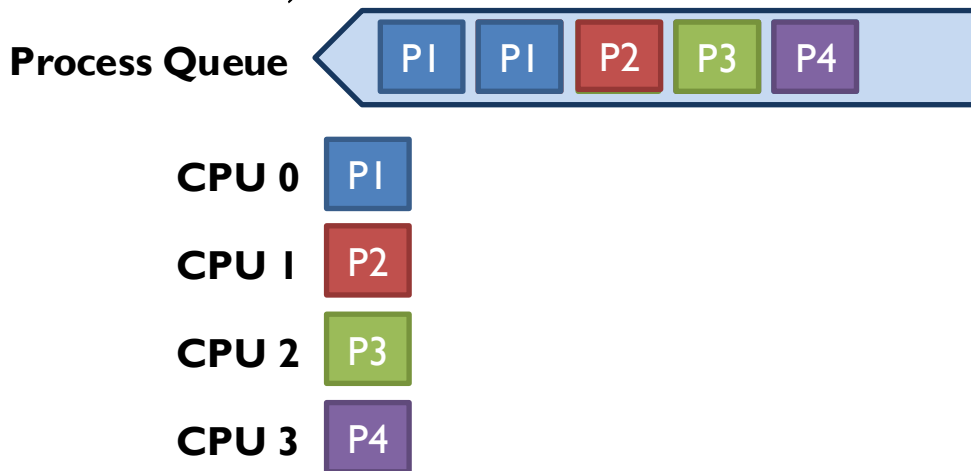
What about systems with multiple CPUs?

- Things get a lot more complicated

Single Queue Scheduling

Single Queue Multiprocessor Scheduling (SQMS)

- Most basic design: all processes go into a single queue
- CPUs pull tasks from the queue as needed
- Good for **load balancing** (CPUs pull processes on demand)



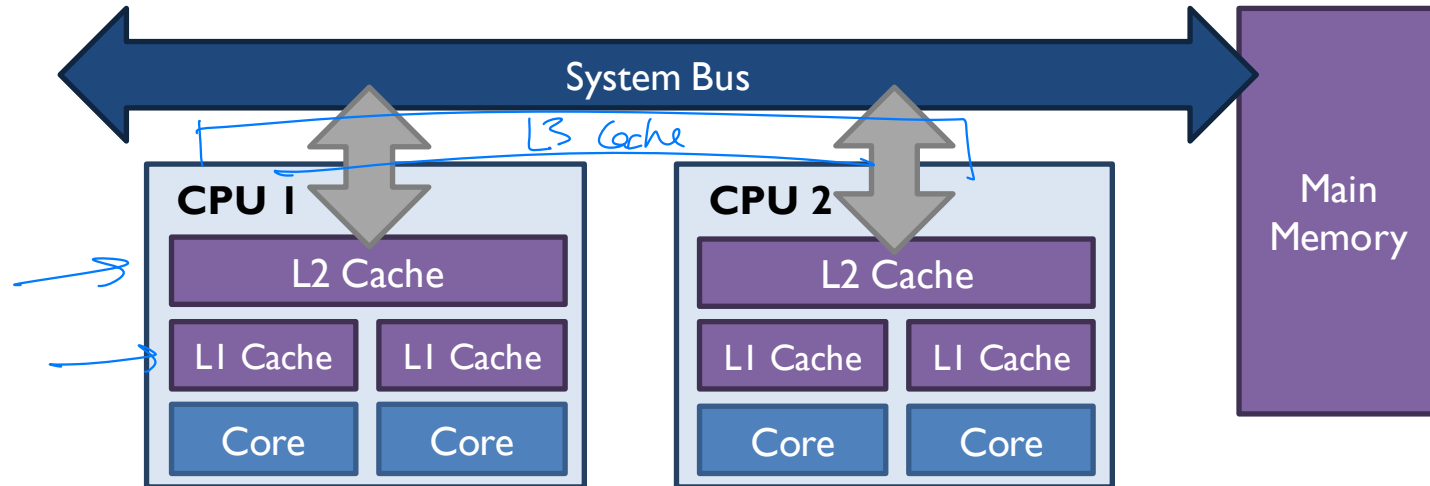
Symmetric Multiprocessing (SMP)

≥2 homogeneous processors

- May be in separate physical packages

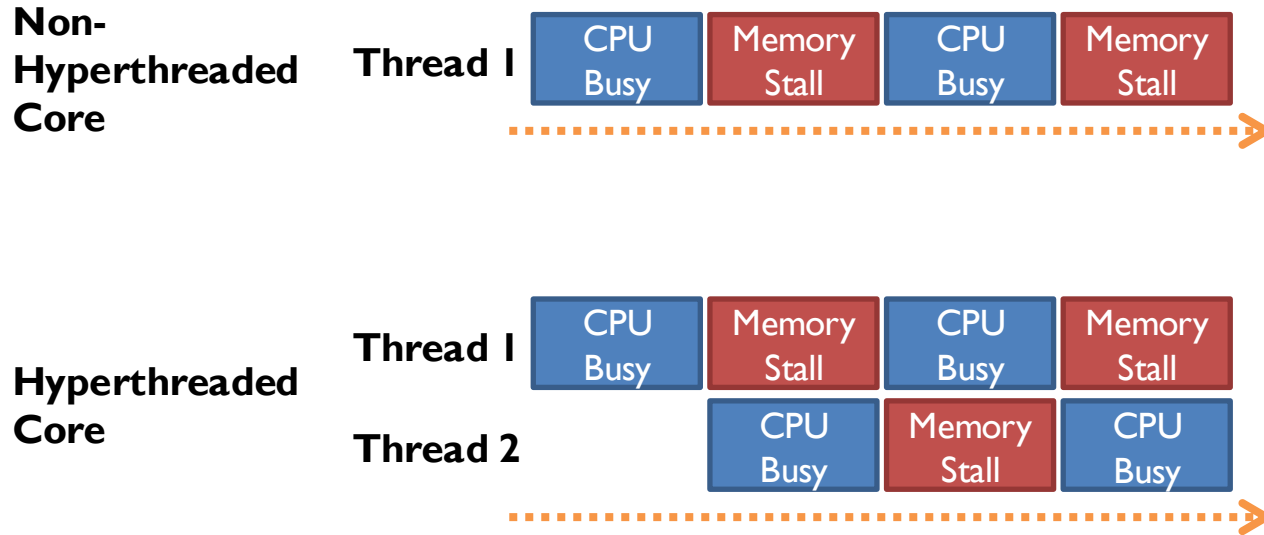
Shared main memory and system bus

Single OS that treats all processors equally



Hyperthreading

Two threads on a single CPU core



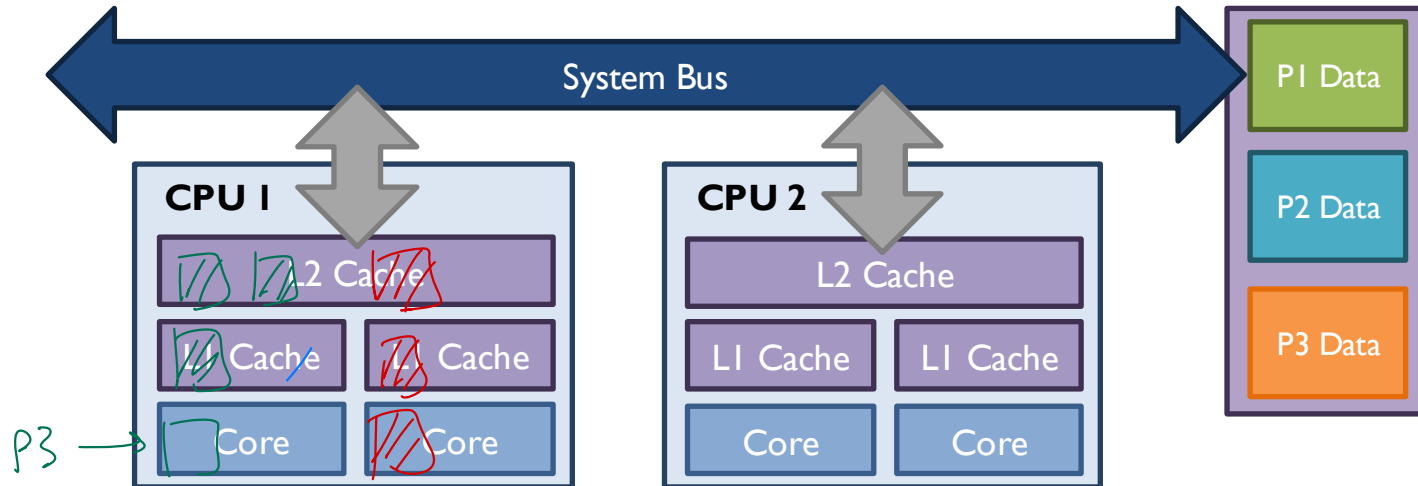
Brief Intro to CPU Caches

Process performance is linked to **locality**

- Ideally, a process should be placed close to its data

Shared data is problematic due to **cache coherency**

- P3 writes variable x, new value is cached in CPU 2
- P2 in CPU 1 reads x, but value in main memory is stale



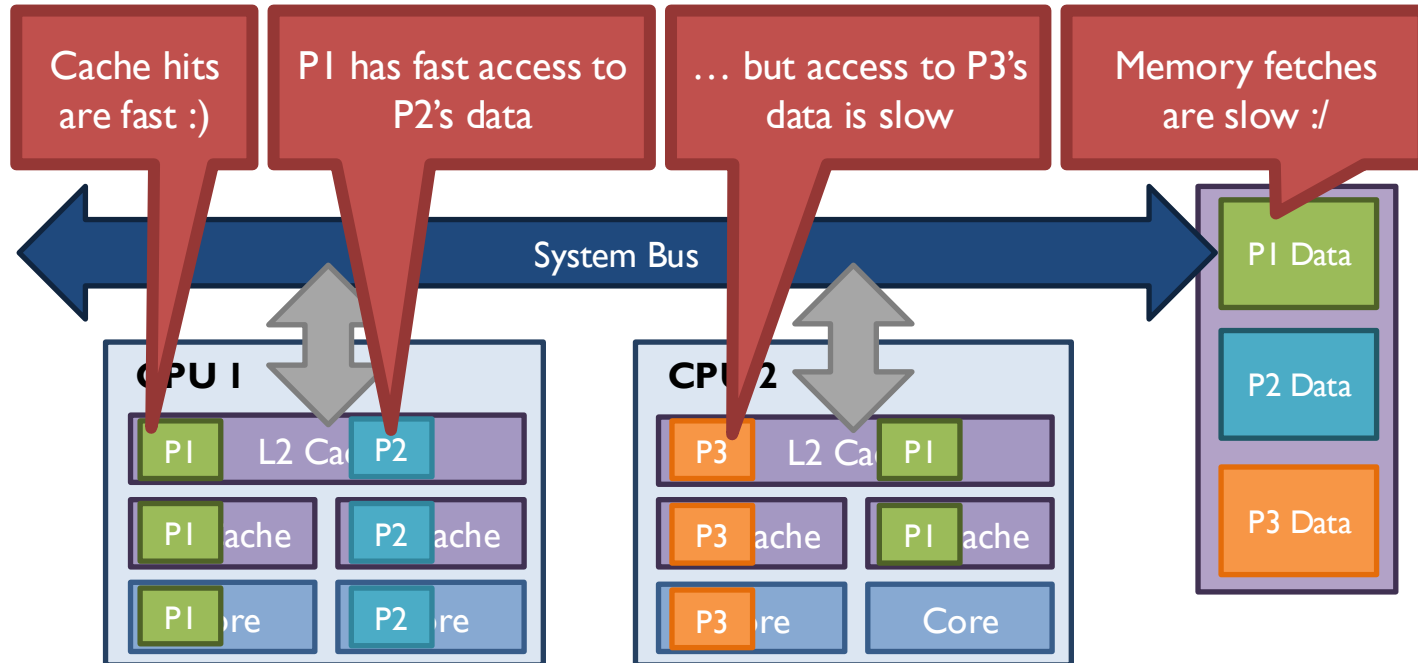
Brief Intro to CPU Caches

Process performance is linked to [locality](#)

- Ideally, a process should be placed close to its data

Shared data is problematic due to [cache coherency](#)

- P3 writes variable x, new value is cached in CPU 2
- P2 in CPU 1 reads x, but value in main memory is stale

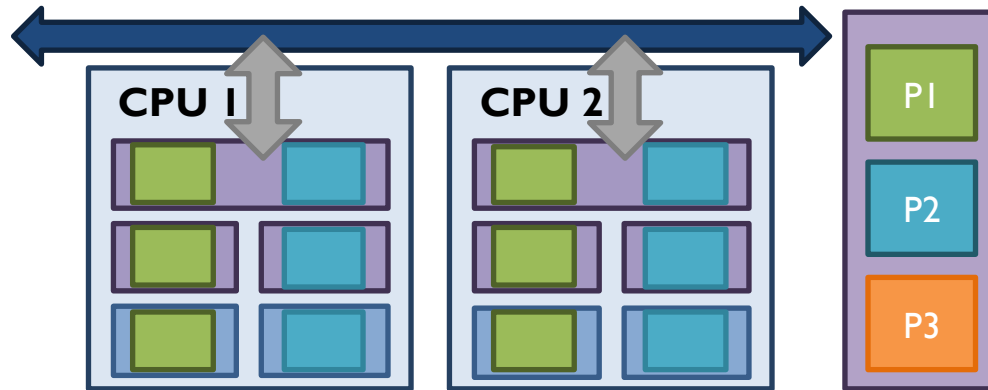


NUMA and Affinity

Non-Uniform Memory Access (NUMA) architecture

- Memory access time depends on the location of the data relative to the requesting process

Cache affinity: try to run processes close to their cached data

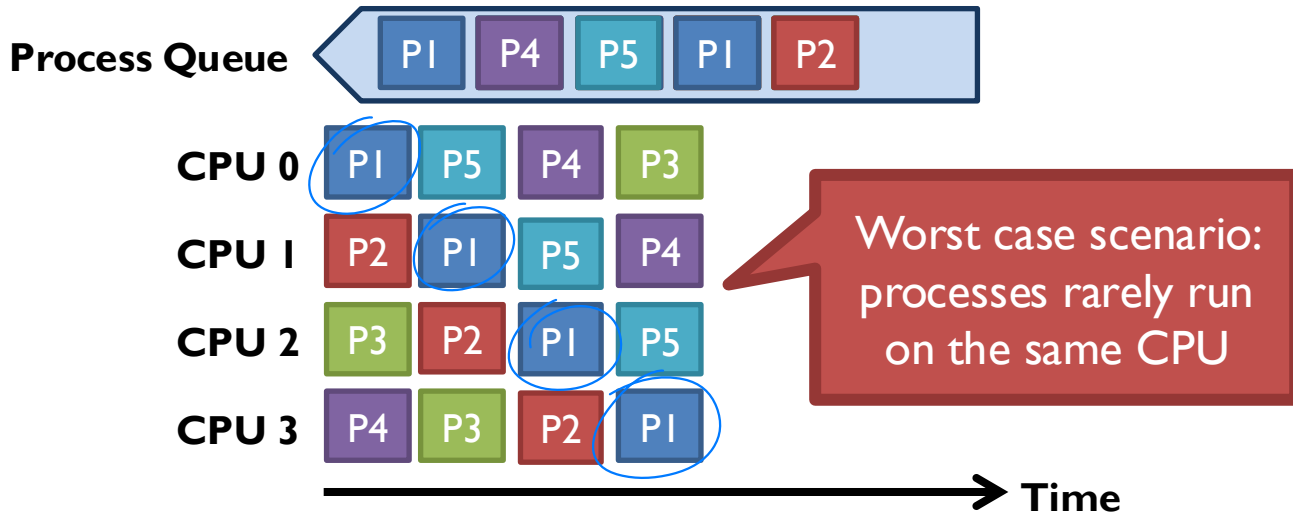


Problems with SQMS

The process queue is a shared data structure

- Necessitates locking, or careful lock-free design

SQMS does not respect cache affinity

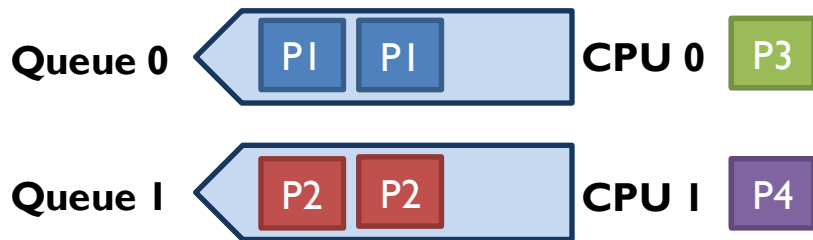


Multi-Queue Scheduling

SQMS can be modified to preserve affinity

Multiple Queue Multiprocessor Scheduling (MQMS)

- Each CPU maintains its own queue of processes
- CPUs schedule their processes independently

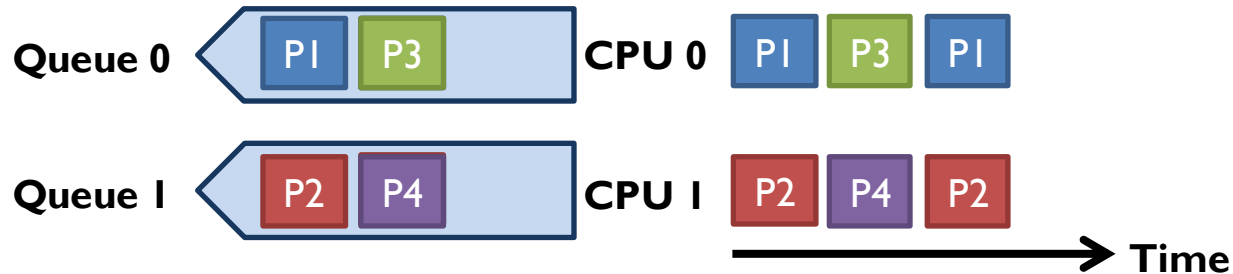


Advantages of MQMS

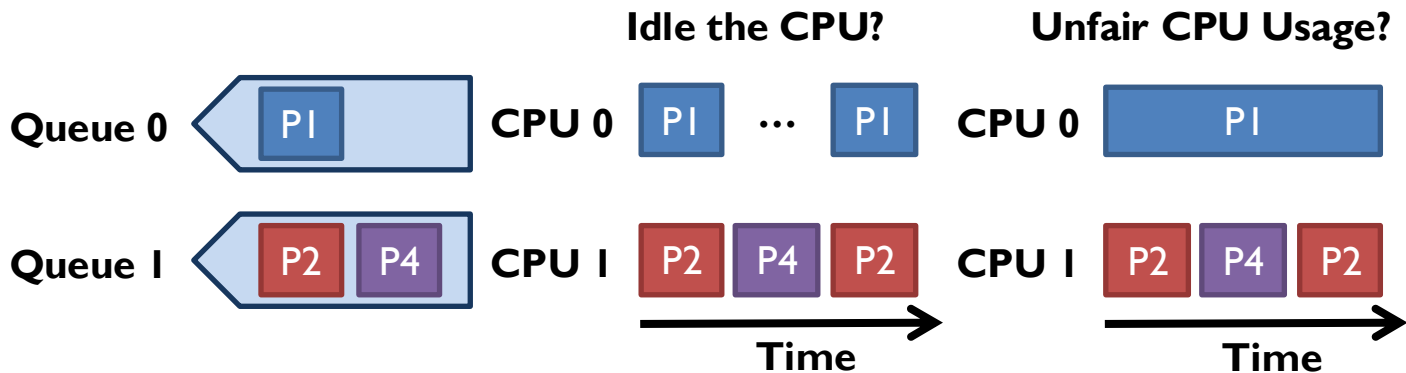
Very little shared data

- Queues are (mostly) independent

Respects cache affinity



Shortcoming of MQMS



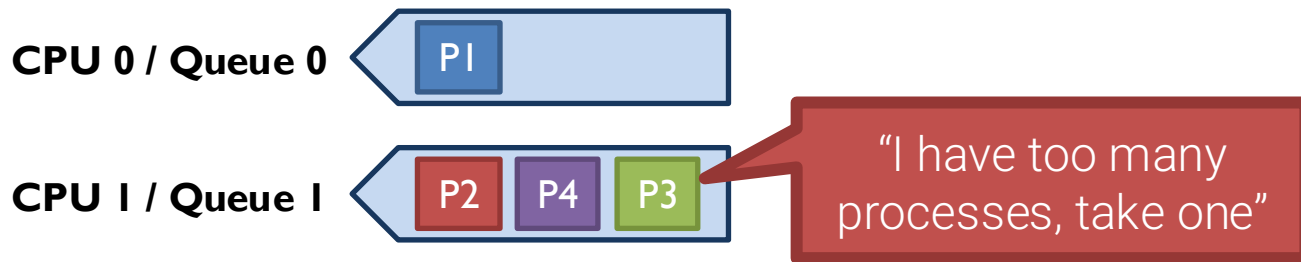
MQMS is prone to **load imbalance** due to:

- Different number of processes per CPU
- Variable behavior across processes

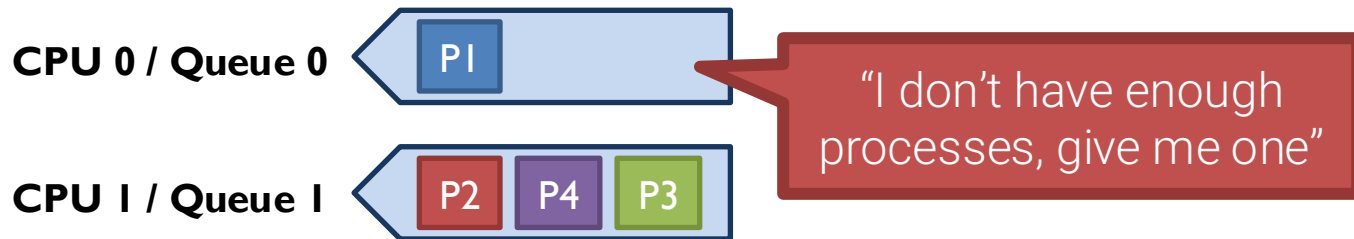
Must be dealt with through **process migration**

Strategies for Process Migration

Push migration



Pull migration, a.k.a. work stealing



Multiprocessor Scheduling

Want **cache locality** to improve performance

Scheduler aims for **cache affinity** (new goal)

Need process migration to resolve load imbalance

Summary

Proportional share (fair) scheduling

- Give processes equal CPU time

Pragmatics of fair scheduling

- Overrun, weighting, reset for sleeping

Multithreaded scheduling: cache locality, affinity, and migration